

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Low (Pseudo-code Writing)

Course Code:	CSA0501	Course Title:	Programming in Java
CO Assessed:	CO1 – Precision (BL3,BL4)	Assessment:	Assessment Tool 3 – Pseudo-code Writing
Weightage:		Total Marks:	100
CO1 Statement:	Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)		
Student Name:	ANUSREE A	Reg. No.:	191126 00 3 3
Section Batch:	/ SLOT B	Date:	27.07.26

Code of Conduct

I ANUSREE A / 1911260033 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 10 questions, each carrying 10 marks. Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 90 minutes.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

91
100

ANNEXURE PSEUDO-CODE WRITING

1. Student Management System [BL3 – Apply]

Write a pseudocode to design a Student class with attributes (name, RollNo, marks) and methods to:

- * Accept student details.
- * calculate average marks.
- * Display result (Pass/Fail).

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP CONCEPT : Encapsulation | Data hiding | classes & objects.

Declare Private name.

Declare Private rollno.

Declare Private marks.

** METHOD acceptDetails () **

Input name

Input roll No

Input marks

** METHOD calculate Average () **

average = marks / 3

Return average

** METHOD displayResult () **

average = calculateAverage ()

2) LIBRARY BOOK MANAGEMENT :

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- * Issue a book (change availability status)
- * Return a book.
- * Display book details using getter methods.

Demonstrate object creation for atleast 3 books and simulate issue / return operations.

OOP CREATION CONCEPT : Encapsulation | Getters & setters | objects.

BEGIN

CLASS Book

Declare Private title.

Declare Private author.

Declare Private ISBN.

Declare Private isAvailable.

** CONSTRUCTOR **

Input title.

Input author.

Input ISBN

Set isAvailable = TRUE

**** GETTER METHODS ****

METHOD get Title ()

Return title

METHOD getAuthor()

Return author

METHOD get ISBN ()

Return ISBN

METHOD get Availability ()

Return isAvailable

**** SETTER METHODS ****

METHOD setTitle (title)

Set title .

METHOD set Author (author)

Set author

METHOD set ISBN (ISBN)

Set ISBN

**** METHOD issue Book() ****

IF isAvailable = TRUE THEN
Set is Available = FALSE

Print "Book issued successfully"

ELSE

Print "Book is not issued successfully"

END IF

** METHOD return Book () **

Set is Available = TRUE

Print "Book returned successfully"

** METHOD issue Book () **

Print Get Title ()

Print Get Author ()

Print Get ISBN ()

Print Get Availability ()

END CLASS

MAIN PROGRAM

Create Book 1

title = "Java Programming"

author = "James"

ISBN = "101".

Create Book 2

title = "Python Basics".

author = "John"

ISBN = "102"

create BOOK 3

title = "C Programming"

author = "Dennis"

ISBN = "103"

Display details of Book 1

Display details of Book 2

Display details of Book 3

Issue Book 1

Display Book 1 details

Return Book 1

Display Book 1 details

Issue Book 2

Display Book 2 details

Return Book 2

Display Book 2 details

END

3) EMPLOYEE SALARY CALCULATION

Write Pseudo code for an employee class with encapsulated attributes (empId, name, basicPay, designation)
Implement methods to : calculate gross salary (basic + HRA + DA)

- * calculate net Salary after deductions (PF, tax)
- * Display Pay Slip.

Apply data hiding so salary components are not directly accessible.

OOP CONCEPTS : Encapsulation | Data hiding | Methods .

CLASS EMPLOYEE

Declare Private emp Id

Declare Private name

Declare Private basic Pay

Declare Private designation.

Declare Private HRA

Declare Private DA

Declare Private PF

Declare Private tax

Declare Private gross Salary.

Declare Private net Salary.

** CONSTRUCTOR **

Input empId

Input name

Input basicPay

Input designation

**** METHOD calculateGrossSalary() ****

HRA = 20% of Basic Pay

DA = 10% of basic Pay

gross Salary = basic Pay + HRA + DA

Return gross Salary

4) **SHAPE AREA CALCULATOR USING POLYMORPHISM**

create a shape base class with an overridden `calculateArea()` method. Apply this to:

* Circle - area using radius.

* Rectangle - area using length and breadth.

* Triangle - area using base and height.

use method overriding to demonstrate runtime polymorphism. call `calculateArea()` for each object.

CLASS Shape

METHOD calculateArea()

DISPLAY "Area of shape"

END METHOD

END CLASS

CLASS Circle EXTENDS Shape

PRIVATE radius

METHOD calculateArea()

area = $3.14 * radius * radius$

DISPLAY "Area of circle" = " , area

END METHOD

END CLASS

CLASS Rectangle EXTENDS Shape

PRIVATE length

PRIVATE breadth

METHOD calculateArea()

area = length * breadth

DISPLAY "Area of Rectangle" = " , area

END METHOD

END CLASS

CLASS Triangle EXTENDS Shape

PRIVATE base
PRIVATE height

METHOD calculateArea()

area = $0.5 * \text{base} * \text{height}$

DISPLAY "Area of Triangle" , area

END METHOD

END CLASS

BEGIN

Shape s1 = new circle()

Shape s2 = new Rectangle()

Shape s3 = new Triangle()

s1. calculateArea()

s2. calculateArea()

s3. calculateArea()

END

5. VEHICLE REGISTRATION SYSTEM

Design a vehicle class with attributes (regno, owner, fueltype). Extend it into :

- * Twowheelers - with engine capacity.

- * Fourwheelers - with seating capacity and AC status.

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

PROTECTED regNo
PROTECTED owner
PROTECTED fuelType

METHOD displayDetails()

DISPLAY "Registration NO: ", regNo

DISPLAY "Owner: ", owner

DISPLAY "Fuel Type: ", fuelType

END METHOD

END CLASS

CLASS TwoWheeler EXTENDS vehicle

PRIVATE engineCapacity

METHOD displayDetails()

DISPLAY "Registration NO: ", regNo

DISPLAY "owner: ", owner

DISPLAY "Fuel Type: ", fuelType

DISPLAY "Engine Capacity: ", engineCapacity, "cc"

END METHOD

END CLASS.

CLASS FourWheeler EXTENDS vehicle

PRIVATE seatingCapacity

PRIVATE AcStatus.

METHOD displayDetails()

METHOD displayDetails()

DISPLAY "Registration NO: ", regNo

DISPLAY "owner: ", owner

DISPLAY "Fuel Type: ", fuelType

DISPLAY "Seating Capacity: ", seatingCapacity

DISPLAY "Ac Status: ", acStatus

END METHOD

END CLASS

BEGIN

TwoWheeler bike = new TwoWheeler()

FourWheeler car = new FourWheeler()

bike.displayDetails()

car.displayDetails()

END

6. BANK ACCOUNT HIERARCHY:

Develop pseudocode for BankAccount superclass
and two subclasses - SavingsAccount (with interest calculation)
and currentAccount (with overdraft facility) analyze and implement
Inheritance of common attributes
Method overriding for withdrawal behaviour

CLASS BankAccount

acc No
balance
holder

METHOD withdraw(amount)

IF amount $\leq$ balance THEN

balance = balance - amount

PRINT "withdrawal successful"

ELSE

PRINT "Insufficient balance"

END IF

END METHOD

CLASS SavingsAccount EXTENDS

BankAccount

METHOD calculateInterest()

interest = balance * 5 / 100

PRINT interest

END METHOD

METHOD withdraw(amount)

IF amount $\leq$ balance THEN

balance = balance - amount

PRINT "withdrawal successful"

ELSE

PRINT "savings limit exceeded"

ENDIF

END METHOD.

7) HOSPITAL PATIENT RECORD SYSTEM

Design a patient base class and analyze how it extends into:

* Inpatient - with ward no, admission date and daily charges.

* outpatient - with appointment for sensitive data (diagnosis, Prescription). override a generateBill() method for each type and compare the billing logic differences.

CLASS PATIENT

PRIVATE PatientId

PRIVATE name

PRIVATE diagnosis

PRIVATE prescription

METHOD generateBill()

Display "General Patient Bill"

CLASS InPatient EXTENDS Patient

PRIVATE ward NO

PRIVATE admission Date

PRIVATE dally charges

PRIVATE days

METHOD generate Bill()

bill = dally charges * days

Display bill

CLASS OutPatient EXTENDS Patient

PRIVATE appointment Date

PRIVATE consultation Fee

METHOD generate Bill ()

bill = consultation Fee

Display bill

MAIN

create Inpatient object

- create outpatient object

call generation Bill () for both objects.

8. E-COMMERCE Product Catalog:

Analyze and design a Product class hierarchy for an e-commerce system

Electronics, with warranty, brand, voltage

Clothing, with size, fabric, gender

Grocery, with expiry date and weight

override an applyDiscount()

CLASS Product

PRIVATE name

PRIVATE price

METHOD applyDiscount()

Return price

CLASS Electronics EXTENDS Product

PRIVATE warranty

PRIVATE brand

PRIVATE Voltage

METHOD applyDiscount()

Apply 10% discount

Return discounted price

CLASS Clothing EXTENDS Product

PRIVATE size

PRIVATE fabric

PRIVATE gender

METHOD applyDiscount()

Apply 20% discount

Return discounted Price

CLASS Grocery EXTENDS Product

PRIVATE expiryDate

PRIVATE weight

METHOD applyDiscount()

Apply 5% discount

Return discounted Price

MAIN

Create Electronics, Clothing and Grocery objects

Store them as Product references

Call applyDiscount()

UNIVERSITY STAFF HIERARCHY

Analyze the design of a Staff base class

extended by Teaching Staff and Non Teaching Staff. Further extend Teaching Staff into Professor and Lecturer (multilevel inheritance). For each level:

- * Identify which attributes should be Private vs Protected
- * Override calculateAllowance() at each level.
- * Analyze how protected access enables inheritance and explain how access modifiers enforce data hiding.

CLASS Staff

PRIVATE StaffId

PRIVATE name

PROTECTED basicSalary

METHOD calculateAllowance()

RETURN basicSalary * 5%

CLASS TeachingStaff EXTENDS Staff

PROTECTED subject

METHOD calculateAllowance()

RETURN basicSalary * 10%

CLASS Professor EXTENDS TeachingStaff

PRIVATE researchArea

METHOD calculateAllowance()

RETURN basicSalary * 20%

CLASS Lecturer EXTENDS TeachingStaff

PRIVATE experience

CLASS NonTeachingStaff EXTENDS Staff

PRIVATE department

METHOD calculateAllowance()

RETURN baseSalary * 8.1.

MAIN

Create Professor object

Create .Lecturer object

Create NonTeachingStaff object

Display professor allowance

Display Lecturer allowance

Display NonTeachingStaff allowance

END

SMART HOME DEVICE CONTROLLER -

Analyze and design a SmartDevice superclass with

Subclasses: SmartLight, SmartThermostat, and SmartLock. override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behaviour. Analyze:

- * How Polymorphism allows a single controller loop to manage all devices.
- * Security implications of data hiding in SmartLock (PIN must never be exposed)

CLASS SmartDevice

METHOD executeCommand(cmd)

Display "Device command"

CLASS SmartLight EXTENDS SmartDevice

METHOD executeCommand(cmd)

Display "Light : " + cmd

CLASS SmartThermostat EXTENDS SmartDevice

METHOD executeCommand(cmd)

Display "Thermostat : " + cmd

CLASS SmartLock EXTENDS SmartDevice

PRIVATE PIN

METHOD executeCommand(cmd)

Display "Lock : " + cmd

Do not display PIN

MAIN

Create SmartDevice List :

SmartLight

SmartThermostat

SmartLock

FOR each device

device . executeCommand ("status")

END FOR

END